ECAI 2025
I. Lynce et al. (Eds.)

© 2025 The Authors.
This article is published online with Open Access by IOS Press and distributed under the terms of the Creative Commons Attribution Non-Commercial License 4.0 (CC BY-NC 4.0).
doi:10.3233/FAIA251326

Adaptive Graph Pruning for Multi-Agent Communication

Boyi Li[a], **Zhonghan Zhao**[b,1], **Der-Horng Lee**[a,*] **and Gaoang Wang**[a, b,*]

[a]Zhejiang University - University of Illinois Urbana-Champaign Institute
[b]Zhejiang University, College of Computer Science and Technology

Abstract. Large Language Model (LLM) based multi-agent systems have shown impressive performance across various fields of tasks, further enhanced through collaborative debate and communication using carefully designed communication topologies. However, existing methods typically employ a fixed number of agents or static communication structures, requiring manual pre-definition, and thus struggle to dynamically adapt the number of agents and topology simultaneously to varying task complexities. In this paper, we propose Adaptive Graph Pruning (`AGP`), a novel task-adaptive multi-agent collaboration framework that jointly optimizes agent quantity (hard-pruning) and communication topology (soft-pruning). Specifically, our method employs a two-stage training strategy: firstly, independently training soft-pruning networks for different agent quantities to determine optimal agent-quantity-specific complete graphs and positional masks across specific tasks; and then jointly optimizing hard-pruning and soft-pruning within a maximum complete graph to dynamically configure the number of agents and their communication topologies per task. Extensive experiments demonstrate that our approach is: **(1) High-performing**, achieving state-of-the-art results across six benchmarks and consistently generalizes across multiple mainstream LLM architectures, with a increase in performance of $2.58\% \sim 9.84\%$; **(2) Task-adaptive**, dynamically constructing optimized communication topologies tailored to specific tasks, with an extremely high performance in all three task categories (general reasoning, mathematical reasoning, and code generation); **(3) Token-economical**, having fewer training steps and token consumption at the same time, with a decrease in token consumption of $90\%+$; and **(4) Training-efficient**, achieving high performance with very few training steps compared with other methods. The performance will surpass the existing baselines after about ten steps of training under six benchmarks. Our code and demos are publicly available at https://resurgamm.github.io/AGP/. s

1 Introduction

LLM-based agents have advanced artificial intelligence by integrating language generation with decision-making and action execution. These agents excel across diverse applications, including code generation [15], embodied agent [32, 43, 46], *etc.*, with a great performance improvement over before. However, the inability of single-agent systems to leverage collective intelligence and foster collaboration limits their effectiveness in solving the intricate tasks present in real-world scenarios. Notably, the collaborative potential of multiple LLM agents, structured within carefully designed communication topologies, has demonstrated superior performance compared to single-agent systems, highlighting the critical role of communication structure in multi-agent intelligence [31, 35, 17]. This has led the current research focus to multi-agent collaborative systems, in which multiple agents engage in iterative interactions within a shared environment, regarding other agents as additional elements of their environment, and continuously refining their strategies and answers through learning from different perspectives of other agents. This process allows them to collectively pursue and achieve a common goal [17].

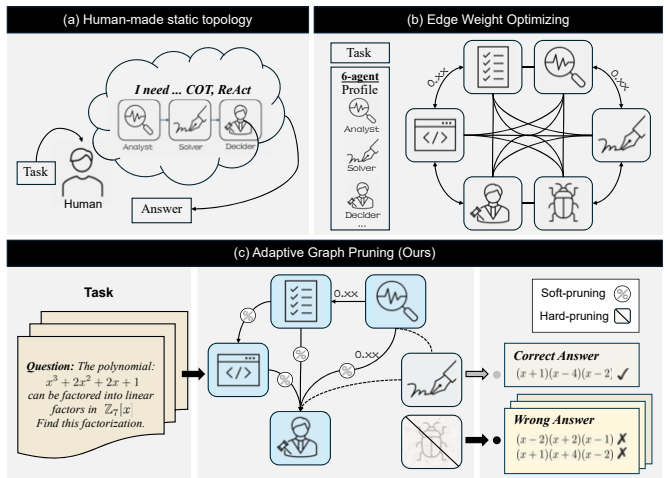


Figure 1: Comparison between existing works and `AGP`. `AGP` produces a dual-pruning method to generate task-adaptive communication topologies.

Despite rapid progress, selecting a communication topology that truly suits each task remains challenging. Most studies still rely on *human-made, static* layouts like chains [34, 15], trees [36, 35, 44, 45], stars [35], complete graphs, or random graphs [28] (Figure 1,(a)). These designs embed strong human assumptions: they may succeed in one domain yet fail in another [48, 39]. Moreover, per-task manual tuning is costly and often overlooks not intuitively relevant agents. More recent work relaxes this constraint by *optimizing edge weights* within a fixed agent pool [48, 39] (Figure 1,(b)). Despite the improvement, the topology remains constrained by a fixed number of predefined agents, which may overlook intuitively relevant agents and limit flexibility and scalability.

We instead pursue a *fully task-adaptive* topology. As Figure 1,(c)

* Corresponding Author. Email: {dhlee, gaoangwang}@intl.zju.edu.cn
[1] Project Leader. Email: zhaozhonghan@zju.edu.cn

shows, activating a key agent (Math Solver) while pruning an irrelevant one (Insect Researcher) yields higher quality in the math domain at lower cost. Experiments further reveal that the most useful agents are sometimes counter-intuitive, underscoring the limits of human design (Section 4.2). To realize this vision, we propose **Adaptive Graph Pruning** (termed as `AGP`), a task-adaptive collaboration framework that jointly optimizes agent quantity (hard pruning) and their communication topology (soft pruning). Because task complexity shapes both the optimal number of agents and their interaction pattern, an adaptive and dynamic topology is essential for consistent, scalable performance.

Our proposed approach incorporates two stages. In **Stage I**, we sample various communication topologies from the agent pool we collected and utilize a graph auto-encoder (GAE) as the soft-pruning module to get the optimal communication topology for the corresponding type of task, composing all the tasks and the corresponding communication topologies as data pairs to obtain the dataset. In **Stage II**, we add the hard-pruning module that shares the same latent space with the soft-pruning module, jointly optimizing hard-pruning and soft-pruning within a maximum complete graph by calculating the relative loss with the task-communication-topology data pair, and finally dynamically configuring the number of agents and their communication topologies per task.

We conduct extensive experiments across six benchmark tasks with twelve baselines, demonstrating that our approach achieves state-of-the-art performance with an average performance improvement of $2.58\% \sim 9.84\%$. Moreover, the comparison with the baselines on training steps, token quantity consumption, and accuracy rate can illustrate that our method is simultaneously training-efficient and token-economical. With only a ten-step training, `AGP` achieves more than a 90% decrease of token consumption in prompting as well as an outstanding performance. Our contribution can be summarized as follows:

- A novel task-adaptive multi-agent collaboration framework, dynamically constructing optimized communication topologies tailored specifically to individual tasks.
- A corresponding two-stage training strategy, jointly optimizing agent quantity (hard-pruning) and communication topology (soft-pruning) for `AGP`.
- Our method delivers state-of-the-art performance while offering excellent inference token economy and high training efficiency.

2 Related Works

LLM Agent Collaboration Topologies. Early work established the superiority of single-agent LLMs for reasoning and planning. These works prompt single-agent LLMs in a designed way, such as chain of thought (CoT) [34], tree of thought (ToT) [36], graph of thought (GoT) [1], complexity-based prompting [10], and self-consistency (SC) [33], to improve text-based reasoning. Subsequent studies showed that *multiple* LLM agents can outperform a single model by combining specialized skills, from majority voting [4] to more intricate interaction schemes [6]. To have further enhanced performance and better agent integration capabilities, recent works explore diverse *pre-defined* or *static* communication topologies: **(1) non-interactive** designs in which agents operate alone without any interactions with others, such as LATM [41], LLM-Blender[20], and LLM-Debate [8]; **(2) chain** structures [28] in ChatDev [27], MetaGPT [15], and L2MAC [14], where each agent receiving and passing information one-by-one; **(3) star** patterns [28] with a central commander or manager that manages other agents in AutoGen [35] and MiniGrid [47]; **(4) tree** hierarchies [28] with a root agent that manages all child agents, such as SoA [19]; and **(5) general graphs** including complete or random variants [28]. **(6) fixed nodes** such as GPTSwarm[48] and G-Designer [39], which optimize edge weights but are not sensitive to the number of nodes, limiting their task adaptation capabilities. However, our Adaptive Graph Pruning (`AGP`) is an *adaptive* topology learner that *jointly* soft-prunes edges and hard-prunes nodes, yielding a task-specific graph that can even shrink below the agent pool size to lower communication cost while boosting accuracy.

Automating Agentic Systems. Research on automating the design of agent-based systems can be grouped into three threads: **(1) Prompt Optimization**, exemplified by PromptBreeder [9], DsPy [21], and EvoPrompt [11]; **(2) Agent Profiling**, including AgentVerse [6], EvoAgent [37], and AutoAgents [3]; and **(3) Inter-agent Communication**, which orchestrates the information flow among agents, as explored by GPTSwarm [48], DyLAN [24], EvoMAC [18], AgentPrune [38], and G-Designer [39]. A recent surge of work pushes this line further by employing search or evolutionary algorithms to explore ever-larger system spaces. For instance, ADAS [16] and AgentSquare [30] automate single-agent design, while AFlow [42] uses Monte Carlo tree search for multi-agent workflow generation, and MaAS [40] searches architectural distributions. Although such approaches can yield strong performance, their freedom to explore vast configuration spaces incurs substantial computational overhead. By contrast, `AGP` sidesteps costly global searches: it adapts communication topologies on the fly over the complete graph, composing task-specific subgraphs that deliver higher efficiency and lower average training and inference costs (*e.g.*, token usage) without sacrificing performance.

Communication Topologies as Graphs. Graphs are a natural abstraction for agent communication and have long been used in MARL [26, 17]. With the advent of LLM agents, researchers recognize that the way multiple agents communicate can be considered from the perspective of the graph, regarding communication topologies as graphs. This approach helps to illustrate the connections between the agents effectively. [17]. This idea is then used for implicit graphs like ChatEval [2], AutoGen [35], and DSPy [21]. Recent works move further to *explicit* graph representations: ChatLLM [12] and DyLAN [24] adopt layered MLP-like graphs; MacNet [28] designs various human-made structures [28]; GPTSwarm [48] and G-Designer [39] regard communication topologies as learnable graphs, designing communication topologies with a fixed number of nodes by just optimizing edge weights. Existing graphs are either predefined or globally optimized once, thus ignoring the per-task difficulty and the heterogeneous usefulness of agents. Our Adaptive Graph pruning (`AGP`), leveraging a *graph pool* collected by the collector, we derive per-task ground-truth sub-graphs and train a dual-pruning GNN that *dynamically* tailors both connectivity and agent subset: the first approach to deliver fully *task-adaptive*, *size-variable* LLM-agent topologies.

3 Method

Adaptive Graph Pruning (termed as `AGP`) first mines high-utility sub-graphs from a fixed pool of heterogeneous LLM-agents and preserves their edge labels and node masks as supervision. The next section sets up notation, casts these ideas in graph-topological terms, and details the multi-agent communication protocol that `AGP` learns

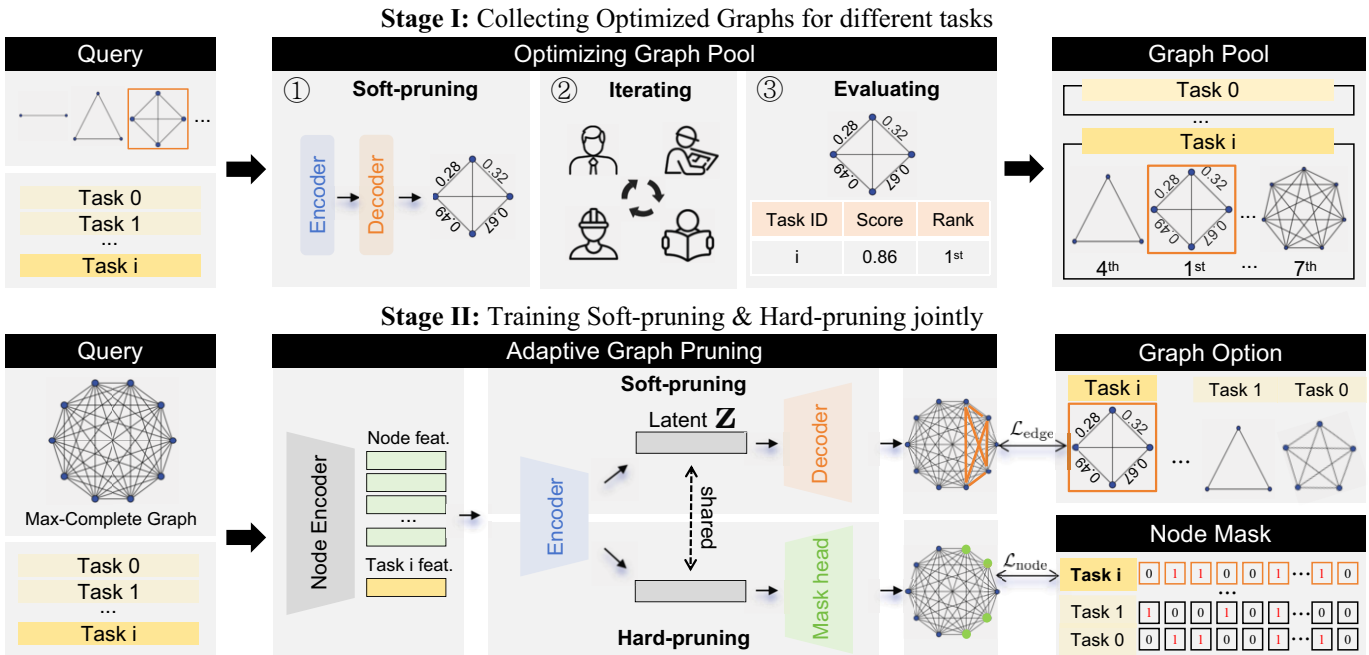


Figure 2: AGP (*Adaptive Graph Pruning*) framework. Stage I mines near-optimal sub-graphs from a heterogeneous agent pool; Stage II trains a joint soft–/hard–pruning network that instantiates an adaptive communication topology for any incoming query $\mathcal{Q}$.

to instantiate for any new query. All notation is listed in Appendix A.2 of [22].

3.1 Preliminaries

Multi-agent system. We represent a set of $N_{\max}$ large-language-model (LLM) agents as a directed graph $\mathcal{G} = (\mathcal{V}, \mathbf{A})$ with $\mathcal{V} = \{v_1, \ldots, v_{N_{\max}}\}$ and $\mathbf{A} \in \{0,1\}^{N_{\max} \times N_{\max}}$, where $A_{ij} = 1$ if and only if messages may flow from v_i to v_j. Each agent is a tuple:

$$v_i = \langle \mathrm{LM}_i,\ r_i,\ s_i,\ \phi_i \rangle, \tag{1}$$

which contains its language backbone LM_i, role/persona r_i, dialogue state (history) s_i, and optional tool plug-ins ϕ_i (*e.g.*, Python, Wolfram).

Design objective. Given a query $\mathcal{Q}$, the team communicates for K rounds and outputs $a^{(K)}$. We aim to select a communication topology $\mathbf{A}$ that maximizes utility while minimizing inter-agent traffic from the sub-graph set $\mathfrak{G}$ of the maximum complete graph with $N_{\max}$ nodes:

$$\min_{\mathbf{A} \in \mathfrak{G}} \Big[-U(\mathbf{A} \mid \mathcal{Q}) + \lambda_c\, C(\mathbf{A}) \Big], \tag{2}$$

where U is task utility (*e.g.*, exact-match accuracy), C counts the number of transmitted tokens, and $\lambda_c > 0$ balances the two.

Two degrees of freedom.

Definition 1 (Soft-pruning). *Assign each potential edge a weight $w_{ij} \in [0,1]$, yielding a weighted adjacency $\mathbf{W}$; small w_{ij} throttles or blocks the bandwidth of $(v_i \to v_j)$.*

Definition 2 (Hard-pruning). *Introduce a binary mask $\mathbf{m} \in \{0,1\}^{N_{\max}}$ where $m_i = 0$ removes agent v_i from the task, producing a node-induced sub-graph $\mathcal{G}[\mathbf{m}]$.*

AGP jointly learns $\mathbf{W}$ and $\mathbf{m}$ so that the resulting graph $\mathcal{G}_{\mathrm{com}} = \mathcal{G}[\mathbf{m}]$, $\mathbf{W}$ is both task-aware and cost-efficient.

3.2 Stage I: Collecting Optimized Graphs

Figure 2 (higher half) shows how AGP samples various communication topologies from the agent pool, collects optimized graphs from the sub-graph set of the maximum complete graph, and finally gets the graph pool. Specifically, **Stage I** contains the following parts:

Agent indexing. We permanently anchor the $N_{\max}$ heterogeneous agents to the complete graph $K_{N_{\max}}$, eliminating permutation ambiguity.

Sampling pool. For every order $i \in [2, N_{\max}]$ the COLLECTOR samples a complete sub-graph K_i by drawing a size-i agent subset uniformly without replacement and inserting them in ascending ID order. Graph orders follow a truncated Gaussian $\mathcal{N}(\mu = N_{\max}/2, \sigma^2)$ until a pool budget B is met.[2]

Task training & scoring. Each sampled graph $\mathcal{G}$ is fine-tuned on its associated task t to obtain $u_t(\mathcal{G}) = \mathrm{Acc}(\mathcal{G}; t) \in [0,1]$. Per task, we keep the top-2 performers $\{\mathcal{G}_t^{(1)}, \mathcal{G}_t^{(2)}\}$ and *lift* them to the $N_{\max}$-node reference frame:

$$\mathbf{A}^{\mathrm{gt}} \in \{0,1\}^{N_{\max} \times N_{\max}}, \quad \mathbf{y} \in \{0,1\}^{N_{\max}}. \tag{3}$$

The pair $(\mathbf{A}^{\mathrm{gt}}, \mathbf{y})$ acts as exact supervision for Stage II.

Corpus statistics. The resulting 460 supervision graphs span three families (Figure 3): 200 *General Reasoning*, 100 *Mathematical Reasoning*, and 160 *Code Generation*.

3.3 Stage II: Training Soft-pruning & Hard-pruning

Figure 2 (lower half) shows how **Stage II** turns the *static* supervision pool harvested in Stage I into a *dynamic* topology generator that adapts to every query. Conceptually, the module follows a two–branch design:

[2] A typical configuration uses $B = 2{,}000$, $\sigma = 2$.

1. **Soft-pruning path** (orange in the figure) learns a *directed, weighted* adjacency that modulates how much information flows between any two retained agents.
2. **Hard-pruning path** (green) learns a binary node mask that decides which agents are retained at all.

Both paths share a common latent representation $\mathbf{Z}$, so the edge weights and node decisions remain mutually consistent.

3.3.1 Network Architecture

To satisfy our conception, the network architecture of `AGP` consists of four modules: a node encoder to embed the agent profile and task into the network; a GCN backbone to calculate the features on the maximum complete graph; an edge-weight head to generate an edge weight matrix for the maximum complete graph; and a node-mask head to generate mask matrix for nodes. Here are the detailed description of these modules.

Node encoder. Each agent profile and the task-specific virtual node (see Stage I) are embedded into $\mathbf{X} \in \mathbb{R}^{N_{\max} \times d}$ via a lightweight Sentence-BERT encoder, giving the model access to both role/tool metadata and query semantics.

GCN backbone. A two-layer Graph Convolutional Network aggregates neighborhood information over the *max-complete* graph, producing latent vectors $\mathbf{Z} = \mathrm{GCN}(\mathbf{X}) \in \mathbb{R}^{N_{\max} \times h}$. This step corresponds to the *Node feat.* and *Latent* $\mathbf{Z}$ blocks in Figure 2.

Edge-weight head. A bilinear projector $\mathrm{Proj}_{\text{edge}} : \mathbb{R}^h \times \mathbb{R}^h \to [0,1]$ maps every ordered pair $(\mathbf{z}_i, \mathbf{z}_j)$ to W_{ij}^{pred}, yielding a dense, directed weight matrix $\mathbf{W}^{\text{pred}} \in [0,1]^{N_{\max} \times N_{\max}}$ with zero diagonal.

Node-mask head. A two-layer MLP produces logits $\mathbf{s} \in \mathbb{R}^{N_{\max}}$ which, after a sigmoid, become continuous masks $\hat{\mathbf{y}} = \sigma(\mathbf{s})$. During inference we threshold $\hat{y}_i \geq 0.5$ to keep only the most relevant agents, as illustrated by the "Node Mask" panel in Figure 2.

3.3.2 Training Details

Why two losses? Edge supervision alone cannot shrink a graph (all nodes remain active), while node-only sparsification collapses the topology into a clique (without edge weighting). Our design, therefore, couples *edge loss* to learn bandwidths and *node loss* to learn minimal agent subsets.

Edge loss (soft-pruning). As shown in below Equation (4), we measure MSE on positive edges (first term) and pushes negatives towards 0 (second term), discounting edges whose endpoints are masked out by $\mathbf{y}$.

$$\mathcal{L}_{\text{edge}} = \frac{1}{\|\mathbf{M}\|_1} \sum_{i \neq j} M_{ij}\, \mathrm{MSE}\big(W_{ij}^{\text{pred}}, A_{ij}^{\text{gt}}\big) + \lambda_{\text{off}} \frac{1}{n(n-1) - \|\mathbf{M}\|_1} \sum_{i \neq j} (1 - M_{ij})\, W_{ij}^{\text{pred}\,2}, \quad (4)$$

where $M_{ij} = y_i y_j$ masks out edges whose *either* endpoint is absent in the ground-truth subgraph.

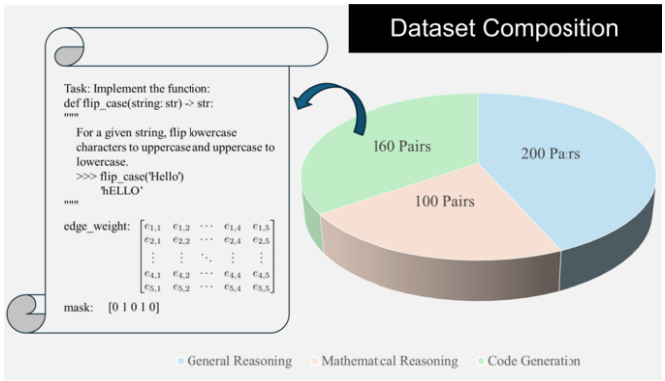


Figure 3: **Stage I** supervision corpus. *Left:* one labeled pair comprising (i) a natural-language task description, (ii) the edge-weight matrix of the *max-complete* graph ($e_{i,j}$ is the weight of $v_i \to v_j$), and (iii) the binary node-mask vector. *Right:* category breakdown of all 460 pairs.

Node loss (hard-pruning). As shown in below Equation (5), we measure BCE between the ground-truth mask matrix and the mask matrix generated. Specifically, let $\hat{\mathbf{y}} = \sigma(\mathbf{s})$:

$$\mathcal{L}_{\text{node}} = \mathrm{BCE}(\hat{\mathbf{y}}, \mathbf{y}) + \lambda_s \langle \hat{\mathbf{y}} \rangle + \lambda_c \frac{1}{n^2} \sum_{i \notin \mathbf{y}} \sum_{j} |W_{ij}^{\text{pred}}|. \quad (5)$$

where the BCE term aligns predicted masks with ground-truth and the sparsity penalty $\lambda_s \langle \hat{\mathbf{y}} \rangle = \frac{1}{n} \sum_{i=1}^{n} \hat{y}_i$. The sparsity penalty $\lambda_s \langle \hat{\mathbf{y}} \rangle$ encourages fewer active agents, while the coherence term penalizes any outgoing weight from ground-truth *absent* nodes, ensuring the two branches agree.

Total objective. $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{edge}} + \beta\, \mathcal{L}_{\text{node}}$ balances structural fidelity and sparsity.

Continuous–Discrete bridge. Both W_{ij}^{pred} and $\hat{y}_i$ use the Gumbel–Sigmoid trick so gradients flow through the discrete sampling process. We anneal at the temperature τ:$1.0 \to 0.1$, so early epochs explore many graphs, while late epochs commit to crisp 0/1 decisions, mirroring the **blurry** $\to$ **sharp** transition sketched in the "Soft-pruning" panel of Figure 2.

3.4 Dataset Composition

Figure 3 visualizes both the *micro* and *macro* structure of our supervision corpus. More data statistics are listed in Appendix A7 [22].

Basic unit (left panel). Each labeled pair is a triple $\langle \text{Task}, \mathbf{A}^{\text{gt}}, \mathbf{y} \rangle$:

- **Task description** : natural-language prompt that an LLM team must solve (*e.g.* implement 'flip_case').
- **Edge-weight matrix** $\mathbf{A}^{\text{gt}} \in \mathbb{R}^{N_{\max} \times N_{\max}}$: the fully specified communication graph lifted to $K_{N_{\max}}$; entry $e_{i,j}$ encodes the ground-truth importance of directed edge $(v_i \to v_j)$.
- **Node mask** $\mathbf{y} \in \{0,1\}^{N_{\max}}$: identifies agents that actually participate in the task.

These rich labels drive the joint soft-/hard-pruning learner described in Section 3.3.

Global statistics (right panel). The corpus totals **460** supervision graphs, drawn from three domains:

Table 1: Performance comparison with single-agent approaches, multi-agent topologies, and AGP. The base LLM for all baselines is `gpt-4o-mini`. Please refer to Appendix A.3 [22] for `gpt-3.5-turbo` performance. We **bold** the best results and underline the runner-ups. "Mul." and "Ada." indicate multi-agent support and task adaptivity, respectively. ×, △, and ✓denote no, partial, and full support.

Method	Mul.	Ada.	MMLU	GSM8K	MultiArith	SVAMP	AQuA	HumanEval	Avg.
Single-agent	×	×	77.81	87.45	96.85	88.26	71.42	87.08	84.81
CoT [34]	×	×	78.43$_{\uparrow 0.62}$	87.10$_{\downarrow 0.35}$	96.31$_{\downarrow 0.54}$	86.24$_{\downarrow 2.02}$	65.00$_{\downarrow 6.42}$	88.13$_{\uparrow 1.05}$	83.54
ComplexCoT [10]	×	×	81.05$_{\uparrow 3.24}$	86.89$_{\downarrow 0.56}$	96.70$_{\downarrow 0.15}$	90.53$_{\uparrow 2.27}$	77.94$_{\uparrow 6.52}$	87.49$_{\uparrow 0.41}$	86.77
SC (CoT×5) [33]	×	×	80.96$_{\uparrow 3.15}$	87.57$_{\uparrow 0.12}$	96.58$_{\downarrow 0.27}$	87.92$_{\downarrow 0.34}$	70.90$_{\downarrow 0.52}$	88.60$_{\uparrow 1.52}$	85.42
MacNet [28]	✓	×	82.98$_{\uparrow 5.17}$	87.95$_{\uparrow 0.50}$	96.03$_{\downarrow 0.82}$	88.06$_{\downarrow 0.20}$	73.20$_{\uparrow 1.78}$	84.57$_{\downarrow 2.51}$	85.47
AgentVerse [6]	✓	×	78.36$_{\uparrow 0.55}$	89.91$_{\uparrow 2.46}$	97.50$_{\uparrow 0.65}$	89.64$_{\uparrow 1.38}$	76.41$_{\uparrow 4.99}$	89.29$_{\uparrow 2.21}$	86.85
MetaGPT [15]	✓	×	–	–	–	–	–	90.93$_{\uparrow 3.85}$	–
LLM-Blender [20]	✓	×	81.22$_{\uparrow 3.41}$	88.35$_{\uparrow 0.90}$	97.29$_{\uparrow 0.44}$	89.52$_{\uparrow 1.26}$	77.28$_{\uparrow 5.86}$	88.80$_{\uparrow 1.72}$	87.08
LLM-Debate [8]	✓	×	81.04$_{\uparrow 3.23}$	89.47$_{\uparrow 2.02}$	97.33$_{\uparrow 0.48}$	91.76$_{\uparrow 3.50}$	78.60$_{\uparrow 7.18}$	88.68$_{\uparrow 1.60}$	87.81
DyLAN [24]	✓	△	79.96$_{\uparrow 2.15}$	89.98$_{\uparrow 2.53}$	97.12$_{\uparrow 0.27}$	88.48$_{\uparrow 0.22}$	75.11$_{\uparrow 3.69}$	90.42$_{\uparrow 3.34}$	86.85
GPTSwarm [48]	✓	△	82.80$_{\uparrow 4.99}$	89.14$_{\uparrow 1.69}$	96.79$_{\downarrow 0.06}$	87.02$_{\downarrow 1.24}$	78.40$_{\uparrow 6.98}$	89.32$_{\uparrow 2.24}$	87.25
G-Designer [39]	✓	△	87.20$_{\uparrow 9.39}$	93.97$_{\uparrow 6.52}$	98.33$_{\uparrow 1.48}$	90.29$_{\uparrow 2.03}$	80.07$_{\uparrow 8.65}$	87.50$_{\uparrow 0.42}$	89.56
MaAS [40]	✓	△	–	92.30$_{\uparrow 4.85}$	98.80$_{\uparrow 1.95}$	–	–	**92.85**$_{\uparrow 5.77}$	–
AGP (Ours)	✓	✓	**87.65**$_{\uparrow 9.84}$	**95.01**$_{\uparrow 7.56}$	**99.44**$_{\uparrow 2.58}$	**92.30**$_{\uparrow 4.04}$	**81.20**$_{\uparrow 9.78}$	90.62$_{\uparrow 3.54}$	**91.04**

- **General Reasoning** *(200 pairs)*: commonsense QA, humanities, social sciences; serves to anchor broad language understanding.
- **Mathematical Reasoning** *(100 pairs)*: arithmetic, algebra, and word-problem reasoning; stresses symbolic manipulation and multi-step deduction.
- **Code Generation** *(160 pairs)*: implement functions from I/O specifications or docstrings; examines the ability to plan, execute, and self-verify algorithmic tasks.

Why this mix? Combining natural-language, numerical, and programmatic tasks forces the topology learner to *generalize* beyond single-domain heuristics: agents required for math (*e.g.* a "Math Solver") differ from those for code generation(*e.g.* a "Programming Expert"), while general reasoning often benefits from broad, low-bandwidth exchanges. The diverse 460-pair corpus, therefore, supplies both breadth and depth to train a robust, query-adaptive pruning policy.

4 Experiments

We evaluate AGP on six public benchmarks that cover three task families: general knowledge, mathematical reasoning, and code generation. The comparison set includes the strongest single-agent prompting techniques (CoT, Complex-CoT, Self-Consistency) and eight multi-agent systems ranging from fixed graphs to adaptive designs and architecture-search.

4.1 Experiment Setup

Tasks and Benchmarks. In order to make an objective evaluation, we evaluate AGP on three categories of datasets corresponding to our training dataset: **(1) For general reasoning**, we use MMLU [13], which is a comprehensive multitask assessment featuring multiple-choice questions across various fields, including humanities, social sciences, hard sciences, and essential subjects like mathematics, US history, computer science, and law; **(2) For mathematical reasoning**, we choose GSM8K [7], a dataset of 8.5K high quality linguistically diverse grade school math word problems. We also select MultiArith [29], SVAMP [25], and AQuA [23], which are diverse in terms of language patterns and problem types, and is used to evaluate the model's ability to solve mathematical word problems; **(3) For code generation**, we evaluate on HumanEval [5], which consists of 164 original programming questions, assessing language understanding, algorithms and simple mathematics, as well as several types of questions for software interviews.

Baselines. To emphasize AGP is generally superior to the existing works. We compare AGP with a total of 12 agentic baselines of the two series: **(1) Single-agent Approaches**: Single-agent LLM, CoT [34], ComplexCoT [10], and Self-Consistency [33]; **(2) Multi-agent Topologies**: MacNet [28]), AgentVerse [6], MetaGPT [15], LLM-Debate [8], LLM-Blender [20], DyLAN [24], GPTSwarm [48], and G-Designer [39].

Implementation Details. We access the GPT through the OpenAI API and mainly test on `gpt-4o-mini`. All models are accessed through APIs with the temperature set to 1. We also set a decision agent to aggregate the history of the dialogue and produce the final solution $a^{(K)}$ with $K = 3$ in all experiments. For all benchmarks, we use $Q \in \{100, 200\}$ queries for optimization. We conduct two evaluations of each baseline under the same benchmark and take the average as the final result. Here are some further details:

- **Mini-batching.** 10 supervision pairs $(\mathbf{A}^{\text{gt}}, \mathbf{y})$ per batch keep GPU memory under 12 GB for $N_{\max}$=16.
- **Optimizer.** Adam, η=10^{-3}, β_1=0.9, β_2=0.95, weight decay 10^{-5}, 10 $\sim$ 20 epochs.
- **Hyper-parameters.** λ_{off}=0.5, λ_s=0.1, λ_c=0.05, β=1.0.
- **Class imbalance.** If fewer than 20% of nodes are active in a batch, BCE switches to Focal Loss (γ=2) to reduce negative overwhelm.

4.2 Quantitative Results

We conduct extensive experiments in performance, training steps, and token consumption across six benchmarks to verify that AGP is:

High-performing. As summarized in Table 1, AGP delivers the strongest overall accuracy (**91.04**%), surpassing the best static-graph competitor G-Designer by **+1.48** and the strongest single-agent baseline (SC) by **+5.62**. It ranks **first on five of the six** task groups, MMLU, GSM8K, MultiArith, SVAMP, and AQuA, achieving per-task gains ranging from **+2.58** (MultiArith) to **+9.84**

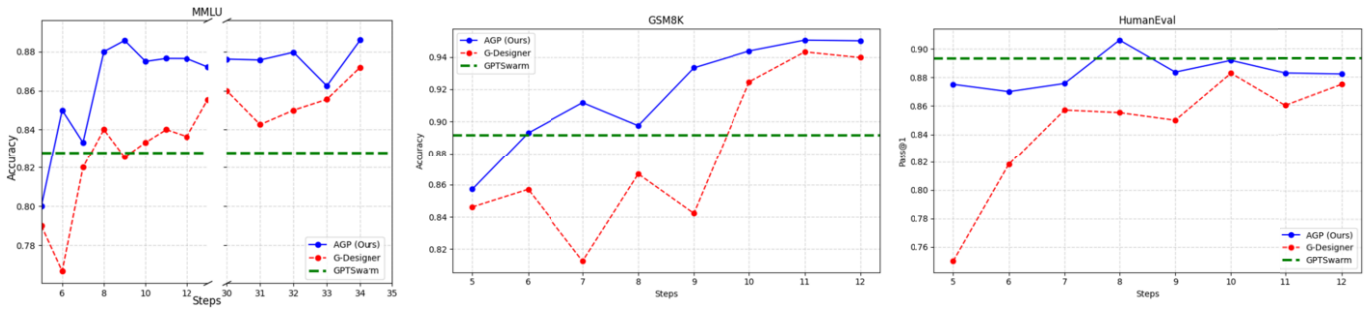


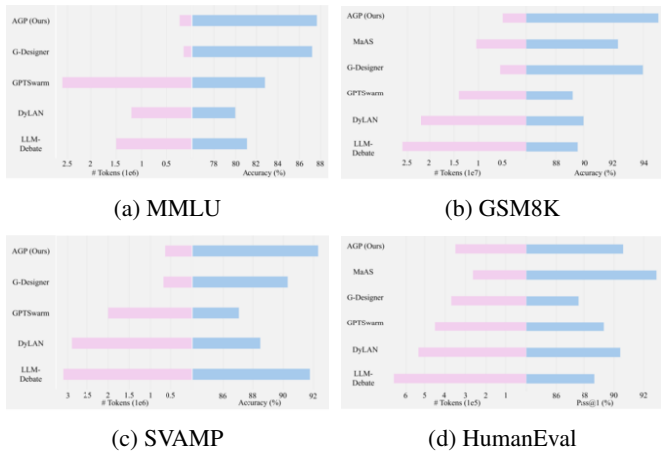
Figure 4: Under MMLU, GSM8k, and HummanEval benchmarks, the curves of the performance of AGP and G-Designer as the number of training steps increases. Starting from the fifth step, there will be an evaluation after each batch is trained.

(MMLU). Even on *HumanEval*, where the code-centric MetaGPT specializes, AGP attains nearly the best score (**90.62**%), while using a single, unified topology learner rather than a domain-specific workflow. The MultiArith result is particularly striking: AGP solves **179 / 180** problems (**99.44**%), indicating that the learned sparse graph preserves crucial numerical-reasoning paths without excessive communication overhead.

Task-adaptive. In order to better explain the task-adaptiveness for multi-agent communication systems, we give the definition: A multi-agent communication system is **completely** task-adaptive if and only if the edge weights and the number of nodes in the communication topology can be changed simultaneously according to different tasks; and a multi-agent communication system is **partially** task-adaptive if and only if either the edge weights or the number of nodes in the communication topology can be changed according to different tasks.

The "Mul." and "Ada." columns reveal that methods able to *both* coordinate multiple agents and tailor the topology per query enjoy the largest gains. AGP is the only fully adaptive system (✓, ✓) compared to all 12 baselines. Among all multi-agent communication systems, AGP is the only one that improves or matches the best baseline across all domains, general knowledge (MMLU +9.84), mathematical reasoning (GSM8K +7.56, SVAMP +4.04, AQuA +9.78), and program synthesis (HumanEval +3.54 over single-agent). Compared with DyLAN and GPTSwarm, which can adjust edges but not shrink the node set, AGP gains **4.19** and **3.79** average accuracy, respectively, illustrating that joint edge–node pruning is key to per-task efficiency. In short, the dual-pruning strategy enables one model to generalize from commonsense QA to symbolic mathematics and near-state-of-the-art code generation with no domain-specific tuning.

Token-economical. Figure 5 contrasts each method's accuracy (blue bars) with its average prompt-token budget per query (pink bars). Three trends emerge clearly. (1) More tokens ⇏ better accuracy for existing systems. LLM-Debate [8], DyLAN [24], and GPTSwarm [48] all follow a "bigger graph, longer prompt" recipe: on MMLU they spend between 1.2×10^6 and 2.6×10^6 tokens yet top out below 83% accuracy, analogous patterns hold on GSM8K, SVAMP, and HumanEval. (2) AGP breaks the token–performance trade-off. With 2.5×10^5 tokens on MMLU, **90% less** than GPTSwarm [48], it raises accuracy from 82.8% to **87.65%** (+4.85). Similar gains aear on GSM8K (+1.04 while cutting tokens by 65%), SVAMP (+5.28, −67% tokens), and HumanEval (+3.12, −22% tokens). (3) Compared to another adaptive graph (G-Designer) AGP is more frugal. G-Designer [39] already trims tokens aggressively, yet AGP still reduces cost on every benchmark (*e.g.* 0.48×10^7 vs. 0.55×10^7 on GSM8K) while adding +1.04 accuracy on average. (4) Even when compared to MaAS [40], a search algorithm that explores agent architecture distributions, our method achieves higher accuracy on GSM8K while reducing prompt-token costs by over half. Although it slightly lags behind MaAS in HumanEval due to MaAS's retention of specific compile-and-test agents, our method still offers a better accuracy-cost trade-off overall.



(a) MMLU (b) GSM8K

(c) SVAMP (d) HumanEval

Figure 5: Visualization of the performance and the number of prompt tokens of different multi-agent communication works across MMLU, GSM8K, SVAMP, and HumanEval benchmarks.

These results confirm that jointly pruning *edges and nodes* not only shortens dialogue transcripts but also steers attention to the most relevant agents. It shows that adaptive graph pruning can provide more accurate and economical solutions in complex settings without the iterative overhead of full architecture searches.

Training-efficient. Figure 4 plots accuracy (or Pass1) against training steps for AGP, G-Designer, and the fixed-graph baseline GPTSwarm. On **MMLU** (left), AGP climbs from 80% to **88%** in only 10 updates, then oscillates narrowly around that peak. G-Designer [39] starts lower (76%), reaches its maximum of 87% at step 34, but never matches our best score. We choose GPTSwarm [48] as our baseline, given its best score of 83%. For **GSM8K** (center), both adaptive methods converge by step 12, but AGP consistently outperforms G-Designer [39] at every checkpoint and reaches the 90% barrier two steps earlier. A similar pattern appears on **HumanEval** (right): our curve remains above G-Designer throughout training and surpasses the GPTSwarm [48] line after the very first evaluation. Taken together, the three curves show that AGP attains higher final accuracy and baseline-beating performance in fewer than ten optimization steps, evidencing markedly better sample- and compute-efficiency during training.

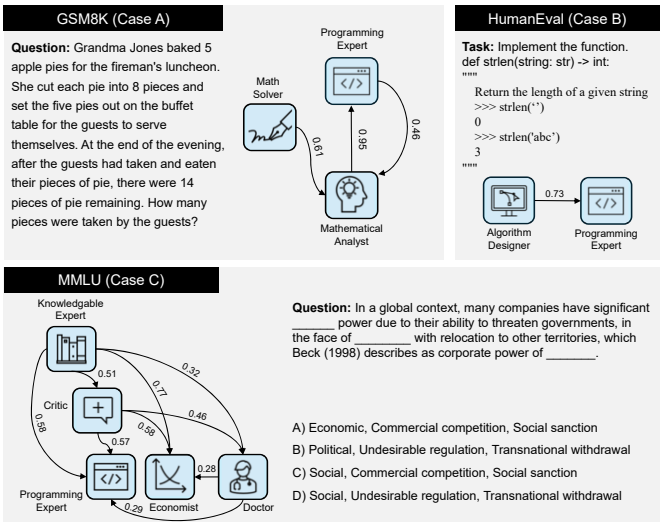


Figure 6: Case study of the communication topologies designed by AGP on GSM8K, HumanEval, and MMLU benchmarks.

4.3 Case Study

A natural question is whether *human-made* topologies, those that wire together exactly the agents a designer deems relevant, must always be the optimal choice. To probe this, we inspect three representative tasks and compare the "intuitive" hand-crafted profiles with the structures selected by AGP. The outcomes fall into three categories, illustrated in Figure 6. More cases are shown in Appendix A.6 [22].

Case A (fully-intuitive). A primary-school arithmetic word problem is solved with a minimal two-agent chain: *Algorithm Designer* produces the reasoning steps and *Programming Expert* executes them. The graph coincides with what a human would have drafted, confirming that AGP does not over-engineer simple tasks.

Case B (partially-intuitive). A simple code development task, `strlen(string)` only requires the core pair of agents, yet the initial pool also contains *Bug Fixer* and *Test Analyst*. AGP prunes these extra nodes, recovering a subset of the human design and achieving the best score with fewer messages than any full four-agent baseline.

Case C (counter-intuitive). A social-and-economic question appears unrelated to medicine, but the model retains both a *Doctor* and the *Programming Expert* alongside a *Statistician*. Empirically, this three-node graph outperforms all hand-picked combinations, suggesting that seemingly "irrelevant" roles can inject orthogonal knowledge or critique that boosts final accuracy.

Together, these examples demonstrate that intuitive human layouts are *not* always optimal. AGP can (1) reproduce them when they suffice, (2) pare them down when redundancy exists, and (3) augment them with non-obvious expertise when the task demands it, underscoring the framework's strong task adaptivity and its ability to uncover useful but non-trivial agent interactions.

4.4 Ablation Study

As shown in Table 2, we disentangle the impact of the two pruning branches. When *both* branches are enabled, AGP sets the state of the art across three representative axes: general knowledge (MMLU **87.65**), grade-school mathematics (GSM8K **95.01**), and function-writing code synthesis (HumanEval **90.62**). Removing **soft-pruning** but still letting the model decide which agents to keep leads to the steepest decline: -4.16 on MMLU, -4.88 on GSM8K, and -4.99 on HumanEval. The reason is that edge weights act as a continuous "bandwidth throttle"; without them, every retained pair either exchanges an uncontrolled flood of messages or is shut off completely, so the system oscillates between information overload and starvation. Conversely, disabling **hard-pruning** while leaving edge weighting intact costs less accuracy, -2.45, -3.45, and -2.21 on the same three tasks, because soft-pruning can still attenuate many noisy links. Yet idle agents continue to produce prompts, wasting tokens and occasionally injecting spurious reasoning paths that drag down precision. In addition, we observe a 22–30% increase in prompt length for the "w/o Hard" variant, confirming the token penalty of carrying dead weight. Taken together, the ablation shows a clear division of labor: **soft-pruning** governs *how loudly* surviving agents speak, whereas **hard-pruning** decides *who gets a voice* in the first place. Only when these two levers operate in concert does the topology become both lean and expressive, unlocking the full benefit of task-adaptive communication.

Table 2: Ablation study of AGP on MMLU, GSM8K, and HumanEval benchmarks.

Variant	MMLU	GSM8K	HumanEval
Vanilla AGP	**87.65**	**95.01**	**90.62**
w/o Soft-pruning	83.49	90.13	85.63
w/o Hard-pruning	85.20	91.56	88.41

5 Limitations

We benchmark AGP with a fixed `gpt-4o-mini` backend in the main paper and `gpt-3.5-turbo` in Appendix A.3 [22]. Whether the dual-pruning policy transfers across other LLM families or model scales remains open. Aside from the node- and edge-analysis and weight in Appendix A.4 A.5 [22], richer quantitative analyses of efficiency are still needed. So far, all results are text-only, applying the adaptive graph to multimodal, temporally extended, embodied-agent tasks (vision–language, tool use, robotics simulations with sensor/action streams) could expose asymmetric bandwidth and other challenges.

6 Conclusion

AGP jointly soft-prunes edges and hard-prunes nodes to yield input-adaptive, variable-size communication graphs, a two-stage process, collecting task-optimized graphs, then training a dual-pruning GNN, lets a single forward pass produce a cost/accuracy-balanced topology. Across six benchmarks spanning knowledge QA, math reasoning, and code synthesis, it attains the best average score (**91.04**%) while cutting prompt tokens by up to **90%** and surpassing strong baselines in fewer than ten optimization steps. Once trained, AGP removes exhaustive graph search and manual workflow design, acting as a plug-and-play module for heterogeneous agent pools and a practical path toward resource-aware multi-agent LLM ecosystems.

7 Acknowledgement

This work was supported by the Zhejiang Provincial Natural Science Foundation of China (LZ24F030005, LD24F020016), the Scientific Research Foundation of Sichuan Provincial Department of Science and Technology, China (2024YFHZ0001), and the Research Fund for International Scientists of National Natural Science Foundation of China (72350710798).

References

[1] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, L. Gianinazzi, J. Gajda, T. Lehmann, M. Podstawski, H. Niewiadomski, P. Nyczyk, and T. Hoefler. Graph of thoughts: Solving elaborate problems with large language models, August 01, 2023 2023.

[2] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu. ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate. *arXiv e-prints*, Aug. 2023.

[3] G. Chen, S. Dong, Y. Shu, G. Zhang, J. Sesay, B. F. Karlsson, J. Fu, and Y. Shi. Autoagents: A framework for automatic agent generation. *arXiv preprint arXiv:2309.17288*, 2023.

[4] L. Chen, J. Q. Davis, B. Hanin, P. Bailis, I. Stoica, M. Zaharia, and J. Zou. Are more llm calls all you need? towards scaling laws of compound inference systems. *arXiv preprint arXiv:2403.02419*, 2024.

[5] M. Chen, J. Tworek, H. Jun, and etc. Evaluating large language models trained on code, July 01, 2021 2021.

[6] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C. Qian, C.-M. Chan, Y. Qin, Y. Lu, R. Xie, Z. Liu, M. Sun, and J. Zhou. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents, 2023.

[7] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv prepring*, abs/2110.14168, 2021.

[8] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. *CoRR*, abs/2305.14325, 2023.

[9] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.

[10] Y. Fu, H. Peng, A. Sabharwal, P. Clark, and T. Khot. Complexity-based prompting for multi-step reasoning. In *The Eleventh International Conference on Learning Representations*, 2022.

[11] Q. Guo, R. Wang, J. Guo, B. Li, K. Song, X. Tan, G. Liu, J. Bian, and Y. Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. *arXiv preprint arXiv:2309.08532*, 2023.

[12] R. Hao, L. Hu, W. Qi, Q. Wu, Y. Zhang, and L. Nie. Chatllm network: More brains, more intelligence, April 01, 2023 2023.

[13] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[14] S. Holt, M. R. Luyten, and M. van der Schaar. L2mac: Large language model automatic computer for extensive code generation. In *The Twelfth International Conference on Learning Representations*, 2024.

[15] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, and C. Wu. Metagpt: Meta programming for multi-agent collaborative framework, August 01, 2023 2023.

[16] S. Hu, C. Lu, and J. Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.

[17] S. Hu, L. Shen, Y. Zhang, and D. Tao. Learning multi-agent communication from graph modeling perspective. *arXiv preprint arXiv:2405.08550*, 2024.

[18] Y. Hu, Y. Cai, Y. Du, X. Zhu, X. Liu, Z. Yu, Y. Hou, S. Tang, and S. Chen. Self-evolving multi-agent collaboration networks for software development. *arXiv preprint arXiv:2410.16946*, 2024.

[19] Y. Ishibashi and Y. Nishimura. Self-organized agents: A llm multi-agent framework toward ultra large-scale code generation and optimization. *arXiv preprint arXiv:2404.02183*, 2024.

[20] D. Jiang, X. Ren, and B. Y. Lin. LLM-blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14165–14178, Toronto, Canada, July 2023. Association for Computational Linguistics.

[21] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

[22] B. Li, Z. Zhao, D.-H. Lee, and G. Wang. Adaptive graph pruning for multi-agent communication. *arXiv preprint arXiv:2506.02951*, 2025.

[23] W. Ling, D. Yogatama, C. Dyer, and P. Blunsom. Program induction by rationale generation: Learning to solve and explain algebraic word problems. *arXiv preprint arXiv:1705.04146*, 2017.

[24] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization. *CoRR*, abs/2310.02170, 2023.

[25] A. Patel, S. Bhattamishra, and N. Goyal. Are nlp models really able to solve simple math word problems? *arXiv preprint arXiv:2103.07191*, 2021.

[26] E. Pesce and G. Montana. Learning multi-agent coordination through connectivity-driven communication. *Machine Learning*, 112(2):483–514, 2023.

[27] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, and M. Sun. Communicative agents for software development, July 01, 2023 2023. 25 pages, 9 figures, 2 tables.

[28] C. Qian, Z. Xie, Y. Wang, W. Liu, Y. Dang, Z. Du, W. Chen, C. Yang, Z. Liu, and M. Sun. Scaling large-language-model-based multi-agent collaboration. *arXiv preprint arXiv:2406.07155*, 2024.

[29] S. Roy and D. Roth. Solving general arithmetic word problems. *arXiv preprint arXiv:1608.01413*, 2016.

[30] Y. Shang, Y. Li, K. Zhao, L. Ma, J. Liu, F. Xu, and Y. Li. Agentsquare: Automatic llm agent search in modular design space. *arXiv preprint arXiv:2410.06153*, 2024.

[31] N. Shinn, B. Labash, and A. Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection. *arXiv preprint*, abs/2303.11366, 2023. doi: 10.48550/arXiv.2303.11366. URL https://doi.org/10.48550/arXiv.2303.11366.

[32] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv e-prints*, art. arXiv:2305.16291, May 2023.

[33] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023.

[34] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models, January 01, 2022 2022.

[35] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework, August 01, 2023 2023.

[36] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, May 01, 2023 2023.

[37] S. Yuan, K. Song, J. Chen, X. Tan, D. Li, and D. Yang. Evoagent: Towards automatic multi-agent generation via evolutionary algorithms. *arXiv preprint arXiv:2406.14228*, 2024.

[38] G. Zhang, Y. Yue, Z. Li, S. Yun, G. Wan, K. Wang, D. Cheng, J. X. Yu, and T. Chen. Cut the crap: An economical communication pipeline for llm-based multi-agent systems. *arXiv preprint arXiv:2410.02506*, 2024.

[39] G. Zhang, Y. Yue, X. Sun, G. Wan, M. Yu, J. Fang, K. Wang, T. Chen, and D. Cheng. G-designer: Architecting multi-agent communication topologies via graph neural networks. *arXiv preprint arXiv:2410.11782*, 2024.

[40] G. Zhang, L. Niu, J. Fang, K. Wang, L. Bai, and X. Wang. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025.

[41] J. Zhang, X. Xu, and S. Deng. Exploring collaboration mechanisms for llm agents: A social psychology view. *arXiv preprint arXiv:2310.02124*, 2023.

[42] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, et al. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.

[43] Z. Zhao, W. Chai, X. Wang, L. Boyi, S. Hao, S. Cao, T. Ye, J.-N. Hwang, and G. Wang. See and think: Embodied agent in virtual environment. *arXiv preprint arXiv:2311.15209*, 2023.

[44] Z. Zhao, K. Chen, D. Guo, W. Chai, T. Ye, Y. Zhang, and G. Wang. Hierarchical auto-organizing system for open-ended multi-agent navigation. *arXiv preprint arXiv:2403.08282*, 2024.

[45] Z. Zhao, K. Ma, W. Chai, X. Wang, K. Chen, D. Guo, Y. Zhang, H. Wang, and G. Wang. Do we really need a complex agent system? distill embodied agent into a single model. *arXiv preprint arXiv:2404.04619*, 2024.

[46] Z. Zhao, W. Zhang, H. Huang, K. Liu, J. Gao, G. Wang, and K. Chen. Rig: Synergizing reasoning and imagination in end-to-end generalist policy. *arXiv preprint arXiv:2503.24388*, 2025.

[47] Z. Zhou, B. Hu, C. Zhao, P. Zhang, and B. Liu. Large language model as a policy teacher for training reinforcement learning agents. *arXiv preprint arXiv:2311.13373*, 2023.

[48] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.